



Fig. 2. Three ways for an intermediary service *IS* to give a client *C* limited access to a target service *TS*: by proxying requests, by having *TS* mint new, restricted credentials for *C*, or by *IS* deriving new macaroons for *C*. Arrows show credential passing and doubled arrows indicate access.

In their expressiveness, and in aspects of their construction, macaroons are related to cascaded proxy certificates [44], restricted delegation [33], active certificates [12], proof-carrying authentication [6], and Grid computing [22], as well as systems for limiting resource consumption [23] and access to networked storage services [3, 25]. Indeed, compared to systems based on authorization logic like SPKI/SDSI [14], it must be stated explicitly that macaroons have no real advantage other than their efficiency and ease of adoption. In particular, in terms of functionality and security, HMAC-based macaroons have the clear disadvantage of being verifiable only by the target service, and by revealing to it certain keys, which prevents useful types of key reuse. (This is detailed in Sections IV-D and V-B and Section V-H outlines a public-key variant of macaroons eliminating this disadvantage.) However, considering the macaroons’ efficiency and flexibility, remarkably few such trade-offs must be made for their practical deployment.

In particular, revocation of authority is a long-standing challenge for credentials-based authorization systems, as detailed in [19, Section 6] and [48, Chapter 9.8]. Broadly, this challenge has been addressed by (i) using very short-lived credentials, (ii) allowing the addition of freshness constraints, (iii) relying on external, authoritative state (such as revocation lists, or epoch counters), and (iv) splitting credentials, and granting authority only to a freshly-gathered collection. Macaroons excel at all four of these revocation strategies: due to their efficiency, and flexibility, macaroons may live only for seconds, and may be split or constrained in any manner, as described in the following sections.

A. Concepts and Constructions

Macaroons operate from a service’s point-of-view, in that they permit a given *target service*, such as a Cloud resource server², to mint a bearer credential that grants authority to access some aspects of the service. A distinguishing characteristic of macaroon credentials is that their bearer can delegate parts of their authority to other principals by deriving new macaroons that both *attenuate* the accessible aspects of the target service and also restrict, via *contextual confinement*, from where, by who, and with what extra evidence, the derived macaroon may be used.

To restrict the authority of a derived macaroon, *caveats* are added and embedded within it. Each macaroon caveat states a

predicate that must hold true for the context of any requests that the derived macaroon is to authorize at the target service—with that predicate evaluated by the target service in the context of each particular request. Notably, as shown in Figure 2, by adding caveats, new, more restricted authorization credentials can be directly derived from macaroons, without proxying requests to the target service, or re-minting new credentials from it. Thus, macaroons can help avoid the latency and overhead, loss of transparency, and trust requirements imposed by the proxying and credential re-minting patterns commonly used in the today’s Cloud.

For example, every macaroon is likely to contain one or more validity-period caveats whose predicates restrict the time (at the target service) during which the macaroon will authorize requests; similarly, each macaroon is likely to have caveats that restrict the arguments permitted in requests. A macaroon may have multiple caveats on the same attribute, such as time, in which case all the caveats’ predicates must hold true in the context of requests.

It is straightforward to add and enforce such *first-party caveats* that confine the context observed at the target service when a macaroon is used. In the Cloud, such restrictions on bearer tokens are sometimes used to reduce the ease by which tokens can be stolen and abused by attackers, while retaining the flexibility and privacy benefits of token-based authorization [5, 13, 17, 31]. Because requests are sent directly from clients (as shown in Figure 2), target services need only check the predicates in such caveats against the context of incoming requests, for example to restrict the source IP address (or, better yet, ChannelID [21]).

However, macaroons’ flexibility stems mostly from *third-party caveats*, their main innovation, which allow a macaroon to specify and require any number of holder-of-key proofs³ to be presented with authorized requests. For example, instead of using time-based validity-period caveats, macaroons might use caveats for a third-party revocation-checking service, and require authorized requests to present fresh proofs discharging those caveats. Such third-party caveats can ensure that each request authorized by a macaroon is subject to any number of extra steps for authentication, authorization, or audit—e.g., to name just a few, checks of network time, authentication at identity providers, auditing and provenance tracking at document repositories, scanning by anti-virus engines, and vetting by a DoS-prevention and risk-analysis service [29].

Even when macaroons are used to enforce complex authorization policies, there is no particular need for end users to be aware of macaroons. In any construction, client-side macaroons will be handled and processed not by users, but by software such as client-side JavaScript in Web browsers. Thus, users can remain equally unaware of macaroons as they are of current Cloud authorization tokens.

Revisiting the example at the start of this section, consider a construction where all macaroons are encoded into “unlisted URLs.” Thus, instead of a HTTP session cookie, the owner may hold a macaroon URL granting them full access to their

²Protocols like OAuth2 [17] use the term *relying party*, not target service.

³In distributed authorization, the term-of-art *holder-of-key proof* is used when a party is authenticated by proving possession of a key, e.g., by signing a protocol message with a private key [30]. Macaroons use holder-of-key proofs extensively, in particular for third-party-caveat discharges.